

Please use Piazza:

<https://piazza.com/class/mk7c83n7ab92h3>

<http://dsg.csail.mit.edu/6.5830/>

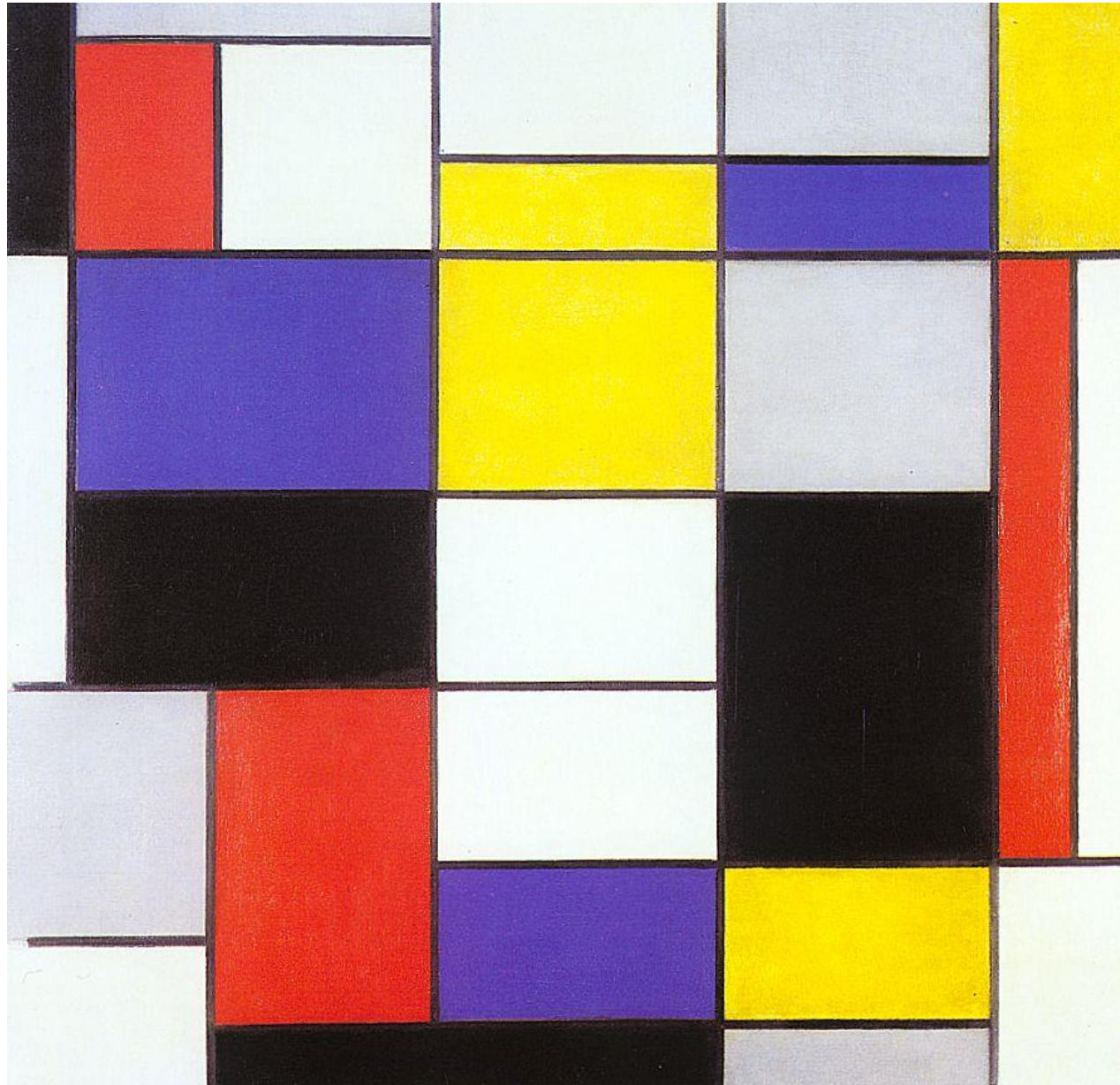
SQL 2

6.5830/1 Lecture 2

2/5/2026

Mike Cafarella

michjc@csail.mit.edu



Summary: Database Systems

- Relational Model + Schema Design
- Declarative Queries
- Query Optimization
- Efficient access and updates to data
 - Recoverability
 - Consistency

Relational Model

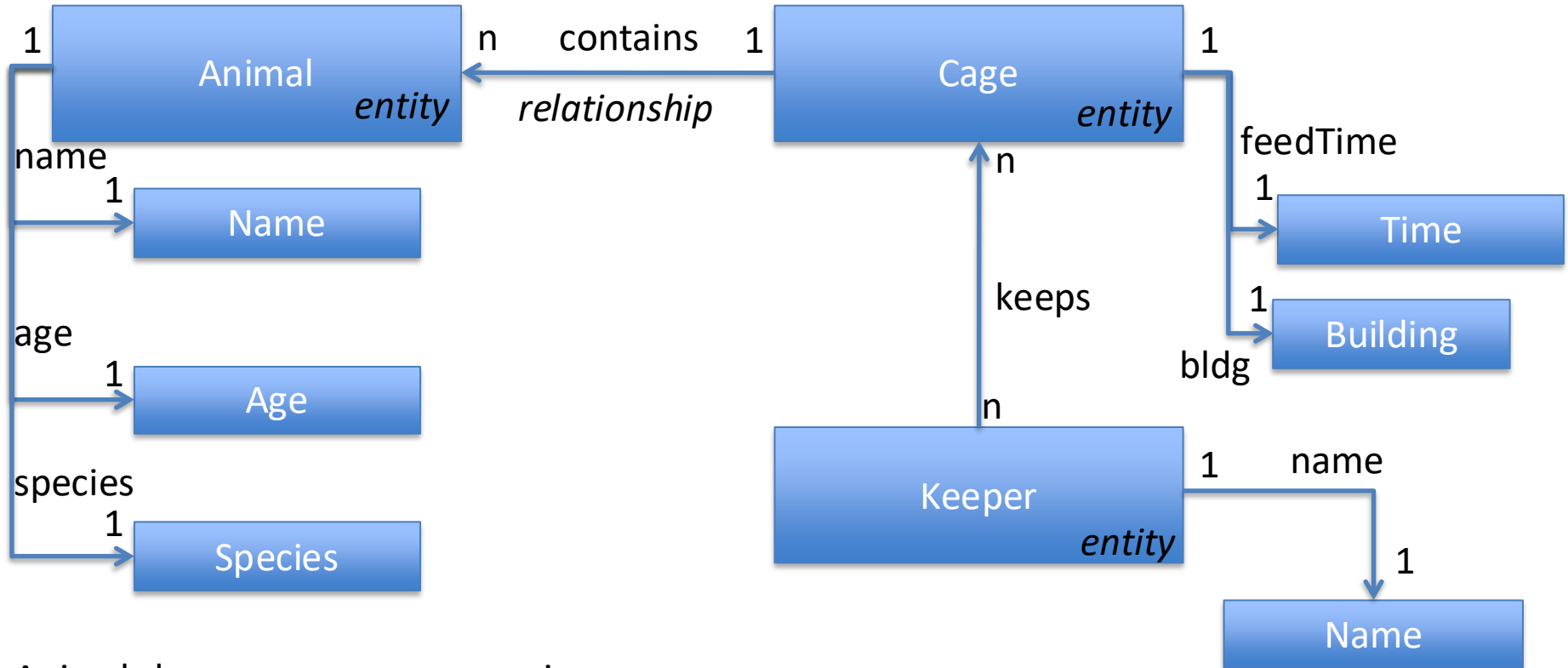
“Those who cannot remember the past are doomed to repeat it”

A Short History Lesson

- **Different Data Models**
 - Hierarchical (IMS/DL1) – 1960's
 - Network (CODASYL) – 1970's
 - Relational – 1970's and beyond
- **Key ideas**
 - Data redundancy (and how to avoid it)
 - Physical and logical data independence
 - Relational algebra and axioms

Recap: Zoo Data Model

Entity Relationship Diagram



Animals have names, ages, species

Keepers have names

Cages have cleaning times, buildings

Animals are in 1 cage; cages have multiple animals

Keepers keep multiple cages, cages kept by multiple keepers

Zoo Tables (aka Relations)

Animals

id	name	age	species	cageno
1	Mike	3	Moose	1
2	Tianyu	12	Giraffe	1
3	Sally	1	Student	2

“Schema”: Field names
& types

Rows, records, or tuples

Cages

no	feedtime	building
1	12:30	1
2	1:30	2

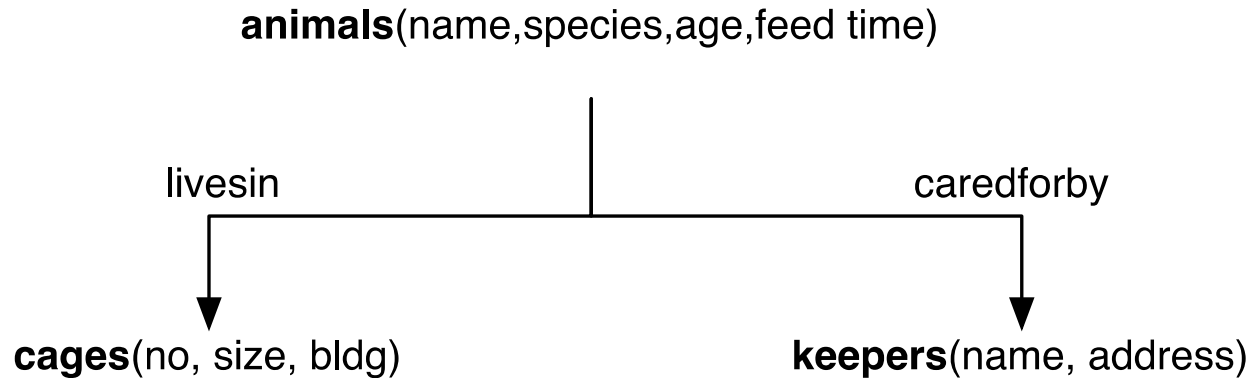
Keepers

id	name
1	Jane
2	Joe

Keeps

kid	cageno
1	1
1	2
2	1

Modified Zoo Data Model



Slightly different compared to last time:

- Each animal in 1 cage, multiple animals share a cage
- Each animal cared for by 1 keeper, keepers care for multiple animals

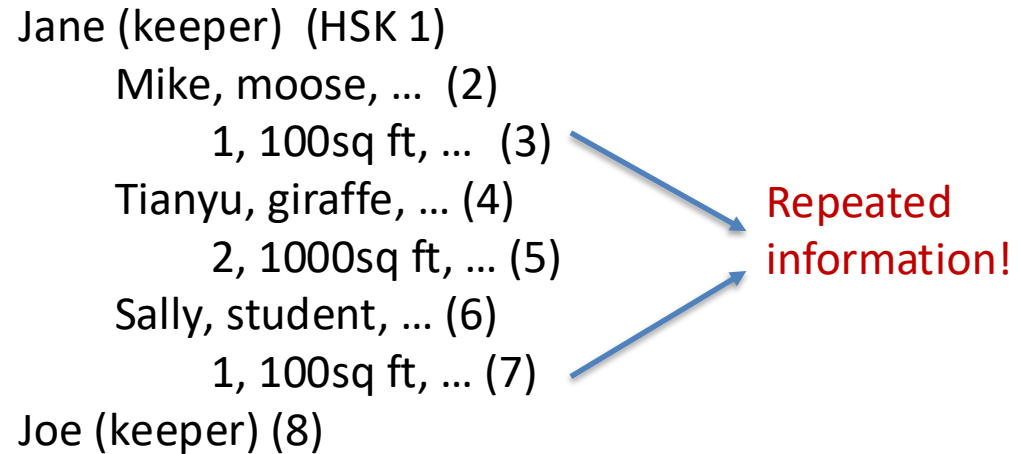
IMS (Hierarchical Model)

- Data organized as *segments*
 - Collection of records, each with same *segment type*
 - Arranged in a tree of segment types, e.g.:



- Segments have different physical representations
 - Unordered
 - Indexed
 - Sorted
 - Hashed

Example Hierarchy



IMS Physical Representation

Keepers segment

A1 Segment

C1 Segment

A2 Segment

C2 Segment

A3 Segment

C3 Segment

Segment Structure

- Each segment has a particular physical representation
 - Chosen by database administrator
 - E.g., ordered, hashed, unordered...
- Choice of segment structure affects which operations can be applied on it

IMS / DL/1 Operations

- **GetUnique** (seg type, pred)
 - Get first record satisfying pred
 - Only supported by hash / sorted segments
- **GetNext** (seg type, pred)
 - Get first or next key in hierarchical order
 - Starts from last GetNext/GetUnique call
- **GetNextParent** (seg type, pred)
 - Same as GetNext, but will not move up hierarchy to next parent
- **Delete, Insert**

Example PL/1 Program #1

Find the cages that Jane keeps

```
GetUnique(Keepers, name = "Jane")
```

Until done:

```
    cageid = GetNextParent (cages).no  
    print cageid
```

Jane (keeper) (HSK 1)

Mike, moose, ... (2)
1, 100sq ft, ... (3)
Tianyu, giraffe, ... (4)
2, 1000sq ft, ... (5)
Sally, student, ... (6)
1, 100sq ft, ... (7)

Joe (keeper) (8)

This iterates through
data underneath
Jane

Implicitly, now
navigating from the
Jane record in
keepers

Example PL/1 Program #2

Find the keepers that keep cage 6

```
keep = GetUnique(keepers)
```

Until done:

```
    cage = GetNextParent(cages, id = 6)
```

```
    if (cage is not null):
```

```
        print keep
```

```
    keep = GetNext(keepers)
```

What's Bad About IMS/PL1?

- Duplication of data w/ non-hierarchical data
- Painful low level programming interface – have to program the search algorithm
- Limited **physical data independence**
 - Change root from indexed to hash --- programs that do GetNext on the root segment will fail
 - Change root from keepers to animals? Also fails.
 - Cannot do inserts into sequential root structure
- Limited **logical data independence**
 - Schemas change, do programs have to?

Logical Data Independence

- Suppose as a cost cutting measure, Zoo management decides a keeper will be responsible for a cage – and all the animals in that cage.

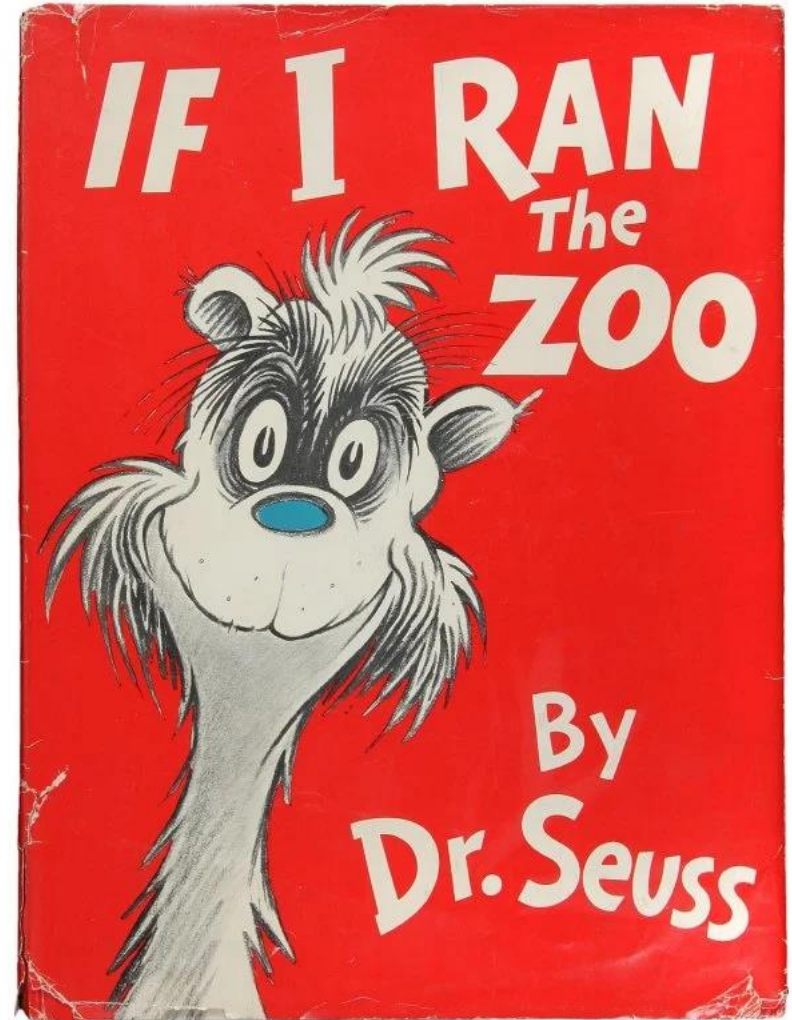


Programs have to change, because the position in the database after a GN/GNP call may not be the same anymore!

Will see how SQL addresses this

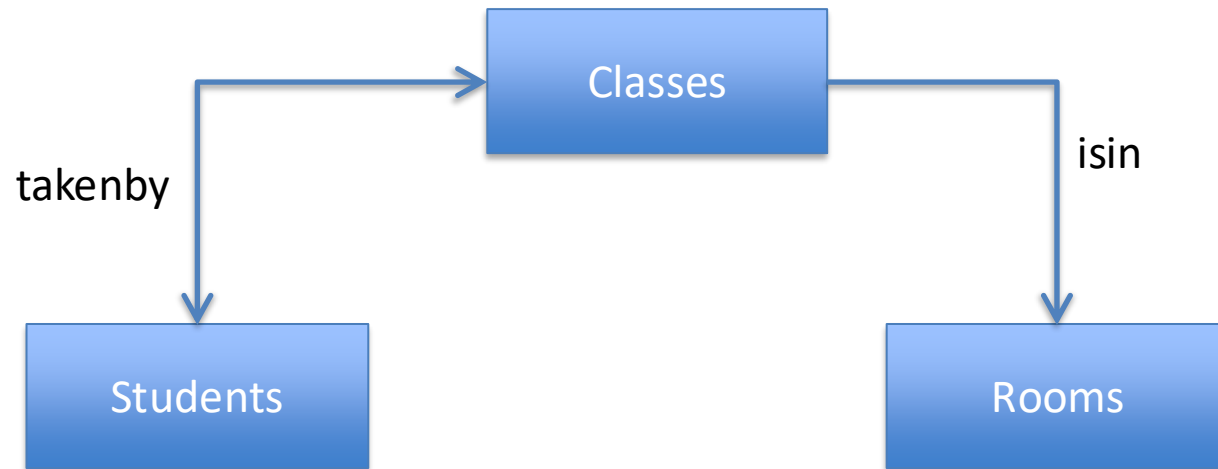
Schemas Change for Many Reasons

- Management decides to have “patrons” who buy cages
 - Need to add a patronid column
- Feds change the rules (OSHA)
 - Keepers can keep at most 2 cages
- Tax rules change (IRS)
- Merge with another zoo



Study break #2

- Consider a course schema with students, classes, rooms (each has a number of attributes)



Classes in exactly one room

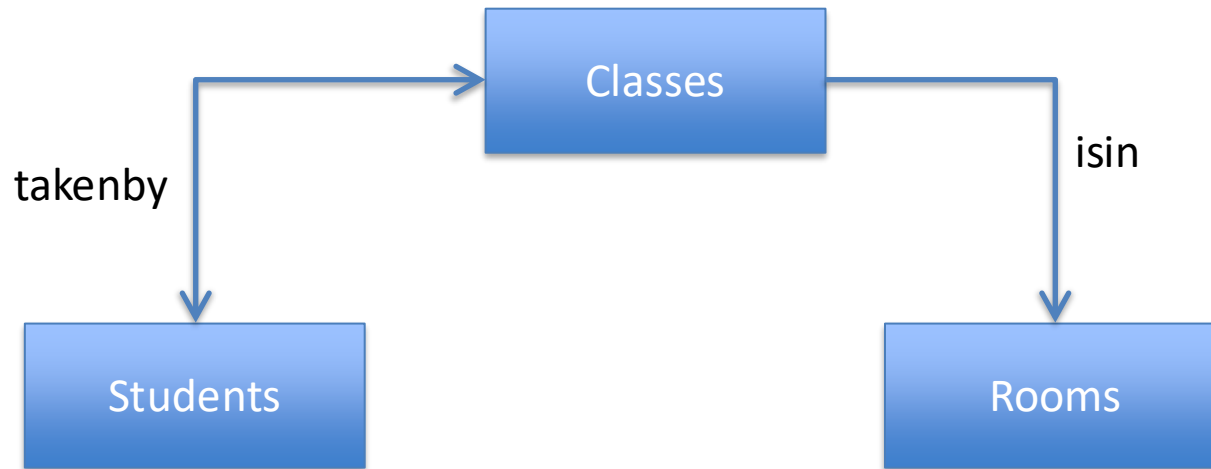
Students in zero or more classes

Classes taken by zero or more students

Rooms host zero or more classes

Questions

1. Describe one possible hierarchical schema for this data
2. Is there a hierarchical representation that is free of redundancy?



Solution

- Many are possible; one example:
 - Classes
 - Students
 - Rooms
- Duplicates data about students,
 - Students take multiple classes, rooms host multiple classes
- Any other arrangement also duplicates data

CODASYL

- **Conference/Committee on Data Systems Languages**
 - Responsible for COBOL
- CODASYL data model developed by consortium of large companies in the 70's
- Designed to address limitations of IMS/PL1
- Graph or network-based data model



The Computer Museum in Boston celebrated cobol's 25th anniversary on May 16, 1985. The cobol tombstone sent to Charles Phillips (see his adjoining article) was presented to the museum. Here surrounding the tombstone (left to right): Ron Hamm, Jack Jones, Jan Prokop, Oliver Smoot, Tom

Rice, Donald Nelson, Grace Hopper, Michael O'Connell, and Howard Bromberg (photo by Lilian Kemp). At the museum's celebration, Bromberg told the following tale of the tombstone.

Example CODASYL Network

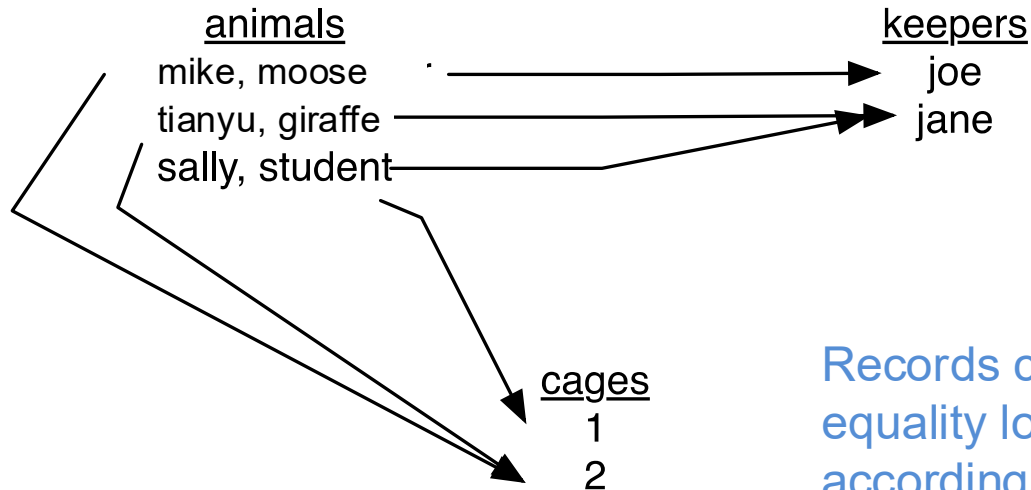
animals(name,species,age)

livesin

caredforby

cages(no, size, bldg)

keepers(name, address)



Records can either be hashes (allowing equality lookup) or sorted ("clustered") according to some key (allowing a range lookup).

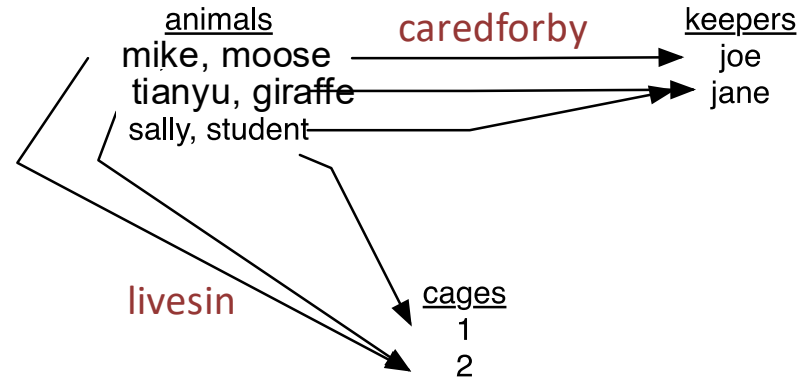
Example: Find Cages Joe Keeps

Find keepers (name = 'Joe')

Until done:

Find next animal in caredforby

Find cage in livesin



- Programming is finding an entry point and navigating around in multidimensional space
 - Each line of code is implicitly at some location in this structure
 - Have to remember where you are

CodasyI Problems

- Incredibly complex —
“Navigational Programming”
- Programs lack physical or logical data independence
 - Can't change schema w/out changing programs;
 - Can't change physical representation either b/c different index types might or might not support different operations
- Some of this could have been fixed by adding a high-level language to CODASYL
- Relational model was a clean-slate approach designed to fix this



Relational Principles

- Simple representation
- Set-oriented programming model that doesn't require "navigation"
- No physical data model description required(!)
 - E.g., no specification of sort orders, hashes, etc

Relational Data Model

- All data is represented as tables of records (*tuples*)
- Tables are unordered sets (no duplicates)
- Database is one or more tables
- Each relation has a *schema* that describes the types of the columns/fields
- Each field is a primitive type -- not a set or relation
- Physical representation/layout of data is not specified (no index types, nestings, etc)

Zoo Tables

Foreign
Keys

Animals

id	name	age	species	cageno	keptby	feedtime
1	Mike	3	Moose	1	1	10:00 am
2	Tianyu	12	Giraffe	1	2	11:00 am
3	Sally	1	Student	2	1	1:00 pm

Primary
Key

Cages

no	building
1	1
2	2

Primary
Key

Keepers

id	name
1	Jane
2	Joe

Primary
Key

Schema: Animals

(id: int,
name: string,
age: int,
species: string,
cageno: int **references** cages.no,
keptby: int **references** keepers.id.
feedtime: time)

Zoo Tables (original schema)

Animals

id	name	age	species	cageno
1	Mike	3	Moose	1
2	Tianyu	12	Giraffe	1
3	Sally	1	Student	2

Primary
Key

Foreign
Key

Cages

no	feedtime	building
1	12:30 pm	1
2	1:30 pm	2

Primary
Key

Keepers

id	name
1	Jane
2	Joe

Primary
Key

Keeps

kid	cageno
1	1
1	2
2	1

Foreign
Key

Foreign
Key

Relational Algebra

- **Projection** ($\pi(T, c_1, \dots, c_n)$)
 - select a subset of columns $c_1 \dots c_n$
- **Selection** ($\sigma(T, \text{pred})$)
 - select a subset of rows that satisfy pred
- **Cross Product** ($T_1 \times T_2$)
 - combine two tables
- **Join** ($\bowtie(T_1, T_2, \text{pred}) = \sigma(T_1 \times T_2, \text{pred})$)
 - combine two tables with a predicate
- Plus set operations (UNION, DIFFERENCE, etc)
- “Algebra” – Closed under its own operations
 - Every expression over relations produces a relation

Join as Cross Product

Animals

name	cageno
Mike	1
Tianyu	1
Sally	2

Cages

no	bldg
1	32
2	36

cageno	no	name	bldg
1	1	Mike	32
1	2	Mike	36
1	1	Tianyu	32
1	2	Tianyu	36
2	1	Sally	32
2	2	Sally	36

Find animals in bldg. 32

```
σ (  
  ⋈(  
    animals,  
    cages,  
    animals.cageno = cages.no  
  ),  
  bldg = 32  
)
```

**Real implementations do not ever
materialize the cross product**

Join as Cross Product

Animals

name	cageno
Mike	1
Tianyu	1
Sally	2

Cages

no	bldg
1	32
2	36

cageno	no	name	bldg
1	1	Mike	32
1	2	Mike	36
1	1	Tianyu	32
1	2	Tianyu	36
2	1	Sally	32
2	2	Sally	36

Find animals in bldg. 32

```
σ (  
  ⋈(  
    animals,  
    cages,  
    animals.cageno = cages.no  
  ),  
  bldg = 32  
)
```

1. animals.cageno = cages.no

Join as Cross Product

Animals

name	cageno
Mike	1
Tianyu	1
Sally	2

Cages

no	bldg
1	32
2	36

cageno	no	name	bldg
1	1	Mike	32
1	2	Mike	36
1	1	Tianyu	32
1	2	Tianyu	36
2	1	Sally	32
2	2	Sally	36

Find animals in bldg. 32

```
σ (  
  ⋈ (  
    animals,  
    cages,  
    animals.cageno = cages.no  
  ),  
  bldg = 32  
)
```

1. animals.cageno = cages.no
2. **bldg = 32**

*Do you think this is how
databases actually
execute joins?*

Relational Identities

- **Join reordering**

- $A \bowtie B = B \bowtie A$
- $(A \bowtie B) \text{ join } C = A \bowtie (B \bowtie C)$

- **Selection reordering**

- $\sigma_1(\sigma_2(A)) = \sigma_2(\sigma_1(A))$

- **Selection push down**

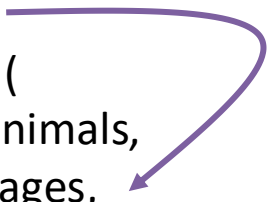
- $\sigma(A \bowtie_{\text{pred}} B) = \sigma(A) \bowtie_{\text{pred}} \sigma(b)$
- σ may only apply to one table

- **Projection push down**

- $\pi(\sigma(A)) = \sigma(\pi(A))$
- As long as π doesn't remove fields used in σ
- Also applies to joins

Push Down Example

```
σ (  
  ⋈ (  
    animals,  
    cages,  
    animals.cageno = cages.no  
  ),  
  bldg = 32  
)
```



```
⋈ (  
  animals,  
  σ (  
    cages,  
    bldg = 32  
  )  
  animals.cageno = cages.no  
)
```

Join Ordering Example

- Find buildings Joe keeps
- SQL

```
SELECT building
FROM cages JOIN keeps ON no = cageno
JOIN keepers on kid = id
WHERE name = 'Joe'
```

*SQL query executor
free to choose either
ordering!*

*Text of SQL query is
not an ordering*

```
⋈ (
  ⋈ (
    cages,
    keeps,
    no = cageno
  ),
  σ (
    keepers,
    name = 'Joe'
  ),
  kid = id
)
```



```
⋈ (
  cages,
  ⋈ (
    σ (
      keepers,
      name = 'Joe'
    ),
    keeps,
    kid=id
  ),
  no = cageno
)
```

*Best
ordering
depends on
sizes of
tables*

*Filtered
keepers may
be **much**
smaller*

Study Break # 2

Schema:

classes: (cid, c_name, c_rid, ...)

rooms: (rid, bldg, ...)

students: (sid, s_name, ...)

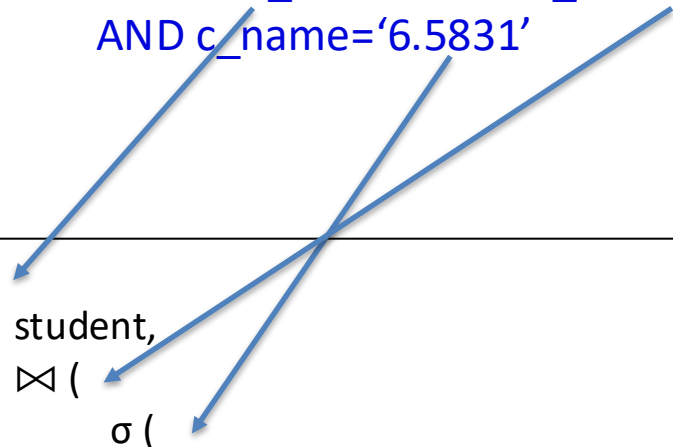
takes: (t_sid, t_cid)

```
SELECT s_name FROM student,takes,classes
WHERE t_sid=sid AND t_cid=cid
AND c_name='6.5831'
```

- Write an equivalent relational algebra expression for this query
- Are there other possible expressions?
- Do you think one would be more “efficient” to execute? Why?

Solution

SELECT s_name FROM student,takes,classes
WHERE t_sid=sid AND t_cid=cid
AND c_name='6.5831'



```
⋈ (
  student,
  ⋈ (
    σ (
      classes,
      c_name = '6.5831'
    ),
    takes,
    t_cid=cid
  ),
  t_sid=sid
)
```

Filtering first is probably a good idea

Filtered table is small, so do join with it and classes first

Will formalize this intuition in a few classes

IMS v CODASYL v Relational

	IMS	CODASYL	Relational
Many to many relationships without redundancy	✗	✓	✓
Declarative, non “navigational” programming	✗	✗	✓

IMS v CODASYL v Relational

	IMS	CODASYL	Relational
Many to many relationships without redundancy	✗	✓	✓
Declarative, non “navigational” programming	✗	✗	✓
Physical data independence	✗	✗	✓

Physical Independence

Can change representation of data without needing to change code

Example:

```
SELECT a.name FROM animals AS a, cages AS c  
WHERE a.cageno = c.no AND c.bldg = 32
```

- Nothing about how animals or cages tables are represented is evident
 - Could be sorted, stored in a hash table / tree, etc
 - Changing physical representation will not change SQL
- No specification of implementation
- Both CODASYL and IMS expose representation-dependent operations in their query API

IMS v CODASYL v Relational

	IMS	CODASYL	Relational
Many to many relationships without redundancy	✗	✓	✓
Declarative, non “navigational” programming	✗	✗	✓
Physical data independence	✗	✗	✓
Logical data independence	✗	✗	✓

Logical Data Independence

- What if I want to change the schema without changing the code?
- No problem if just adding a column or table
- *Views* allow us to map old schema to new schema, so old programs work
 - *Even when changing existing fields*

Key Idea: View

- View is a logical definition of a table in terms of other tables
- E.g., a view computing animals per cage

```
CREATE VIEW cage_count as  
(SELECT cageno, count(*)  
FROM animals JOIN cages ON cageno=no  
GROUP by cageno  
)
```

This view can be used just like a table in other queries

Views Example

- Suppose I want to add multiple feedtimes?
- How to support old programs?
 - Rename existing animals table to animals2
 - Create feedtimes table
 - Copy feedtime data from animals2
 - Remove feedtime column from animals2
 - Create a view called animals that is a query over animals2 and feedtimes

```
CREATE VIEW animals as (  
  SELECT name, age, species, cageno,  
    (SELECT feedtime FROM feedtimes WHERE animalid = id LIMIT 1)  
  FROM animals2  
)
```

Summary: IMS v CODASYL v Relational

	IMS	CODASYL	Relational
Many to many relationships without redundancy	✗	✓	✓
Declarative, non “navigational” programming	✗	✗	✓
Physical data independence	✗	✗	✓
Logical data independence	✗	✗	✓

Next time: Fancy SQL